

Algorithms and Data Structures 2.

—

Breadth First Search – BFS

Harrach Nóra - harrachnora@inf.elte.hu

Practice 3

Table of Contents

Exercise

Breadth-First Search - BFS

Transitive Closure

Exercise

Design an algorithm which calculates the in-degrees and out-degrees of all the vertices of a graph given in (a) adjacency matrix representation or (b) adjacency list representation.

These values should be given in two arrays *In/1* and *Out/1*.

a)

Exercise

Design an algorithm which calculates the in-degrees and out-degrees of all the vertices of a graph given in (a) adjacency matrix representation or (b) adjacency list representation. These values should be given in two arrays *In/1* and *Out/1*.

b)

Table of Contents

Exercise

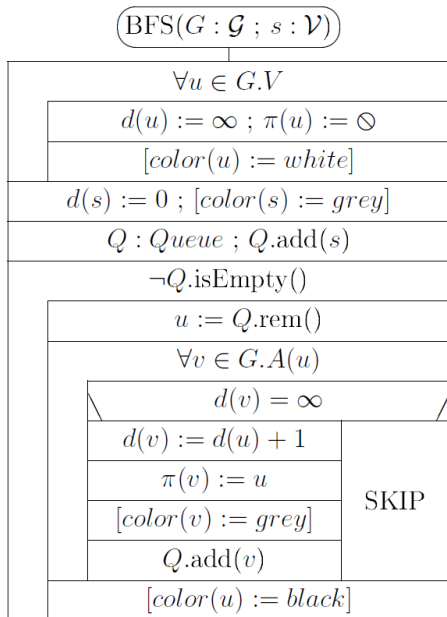
Breadth-First Search - BFS

Transitive Closure

Breadth-first search

- ▶ Breadth-first search (or BFS) will be considered on both undirected or directed graphs.
- ▶ We will fix a source vertex $s \in G.V$.
- ▶ We will find the shortest path to each other vertex available from s .
- ▶ If there are more than one shortest paths, then we only compute one of them.

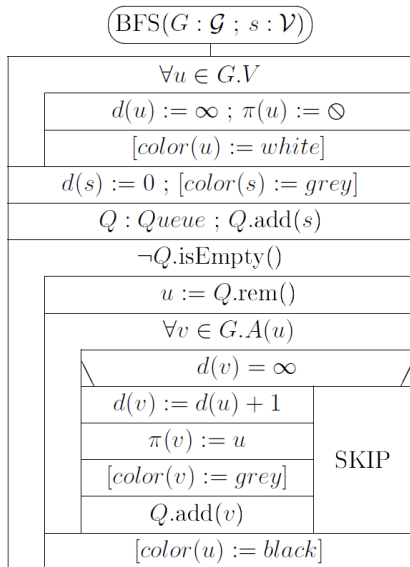
Breadth-first search



Breadth-first search

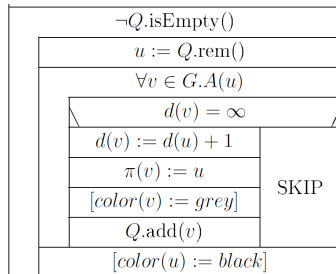
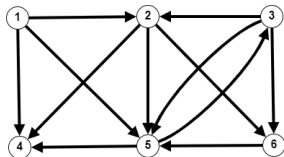
- ▶ For every vertex u we will have three labels:
 - ▶ $d(u)$ the length of the shortest $s \rightsquigarrow u$ path found
 $d(u) = \infty$ means no such path has been found so far
 $d(u) = 0$ is only true for $u = s$
 - ▶ $\pi(u)$ the parent vertex of u on the $s \rightsquigarrow u$ path found.
 $\pi(u) = \emptyset$ means that we have not found such path or $u = s$
 - ▶ $color(u)$ can be omitted from the program (useful for illustration)
 - ▶ *white* if it has not been found yet
 - ▶ *grey* if it has been found but not yet processed
 - ▶ *black* if it has already been processed
- ▶ In the first loop of the algorithm these labels are set:
 $\forall u \in G.V: d(u) = \infty, \pi(u) = \emptyset$ and $color(u) = white$
for s source: $d(s) = 0, \pi(s) = \emptyset, color(s) = grey$

Breadth-first search



- ▶ s is added to queue Q
- ▶ Main loop: in every iteration a vertex u is removed from Q
- ▶ and "expanded", i.e. all its neighbours are considered, and if they have not been found before ($d(v) = \infty$) then their d , π and $color$ labels are set, and they are added to Q
- ▶ Expanded vertices become black.

Example for BFS

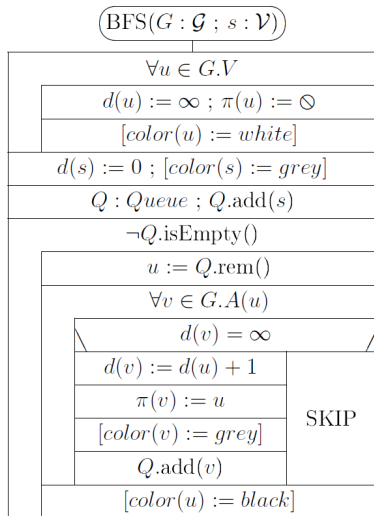


changes of d						ex- panded vertex : d	$Q :$ Queue $\langle \quad \rangle$	changes of π					
1	2	3	4	5	6			1	2	3	4	5	6
final d and π values													

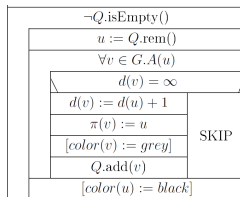
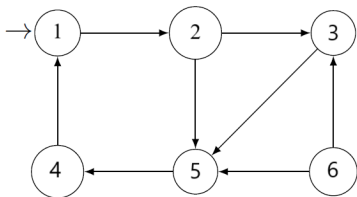
Properties

- ▶ Edges $(\pi(u), u)$ form the **breadth-first tree** or **shortest paths tree** with source vertex s of G .
- ▶ The set of vertices with d value k are the vertices on level k of the breadth-first tree. These are the vertices of G at distance k from source vertex s .
- ▶ During the algorithm when a vertex v with value $d(v) = k$ is being expanded (or processed), then all the vertices in the queue Q have d value k or $k + 1$, and the ones with d value k come first.
- ▶ When the last vertex with d value k has been expanded, then the set of vertices in Q are exactly the set of vertices with d value $k + 1$, i.e. the vertices at distance $k + 1$ from s .

Properties of Breadth-first search

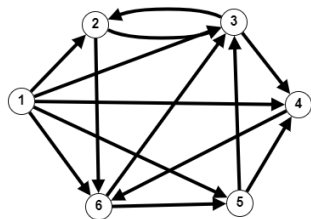


Example for BFS



changes of d						ex- panded vertex : d	$Q :$	changes of π					
1	2	3	4	5	6		Queue	1	2	3	4	5	6
							$\langle \quad \rangle$						
						final d and π values							

Example for BFS

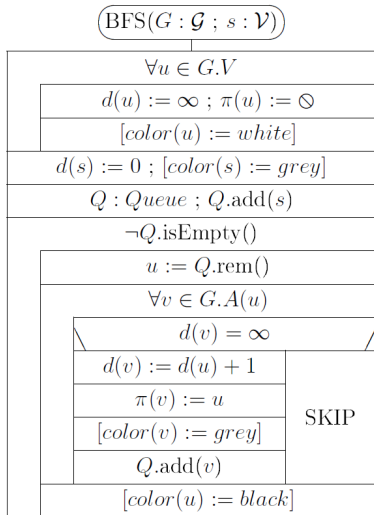


changes of d						ex- panded vertex : d	Q : Queue $\langle \quad \rangle$	changes of π					
1	2	3	4	5	6			1	2	3	4	5	6
						final d and π values							

Optional assignment

Let us suppose that we have run procedure $BFS(G, s)$. Write an algorithm $printShortestPathTo(v)$ which prints the shortest path $s \rightsquigarrow v$ that has been calculated by BFS if v is available from vertex s , and prints " v is not available from s " otherwise. Both recursive and non recursive algorithms can be given to solve this problem.

Efficiency of BFS



- ▶ $n = |G.V|$ and $m = |G.E|$
- ▶ Initializing loop iterates n times.
- ▶ The main loop iterates as many times as the number of vertices available from s (which is ≥ 1 and $\leq n$).
- ▶ For the inner loop the max number of iterations is m or $2m$ (directed, undirected), while the min is 0.
- ▶ Time complexity:
 $MT(n, m) \in \Theta(n + m)$,
 $mT(n, m) \in \Theta(n)$

Table of Contents

Exercise

Breadth-First Search - BFS

Transitive Closure

Transitive closure

For each pair of vertices (u, v) of a network/graph, we want to determine whether there is any path from u to v or not.

We are interested neither in the path nor in its length. For this purpose we introduce the following notion.

Definition: Given graph $G = (V, E)$ its **transitive closure** is relation $T \subseteq V \times V$ where:

$$(u, v) \in T \iff \exists \text{ path from vertex } u \text{ to vertex } v \text{ in graph } G.$$

Transitive closure

$(u, v) \in T \iff \exists$ path from vertex u to vertex v in graph G .

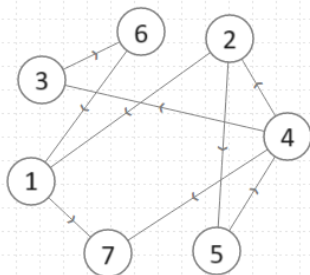
If the vertices of the graph are represented by indices $1, 2, \dots, n$, then we can represent the transitive closure of G with matrix

$$T/1 : \mathbb{B}[n, n]$$

where

$$T[i, j] = 1 \iff \exists \text{ path } i \rightsquigarrow j \text{ in the graph.}$$

Example



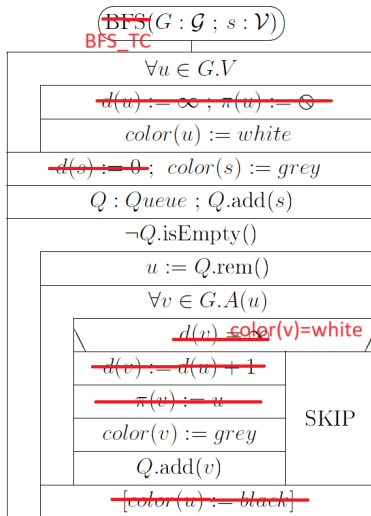
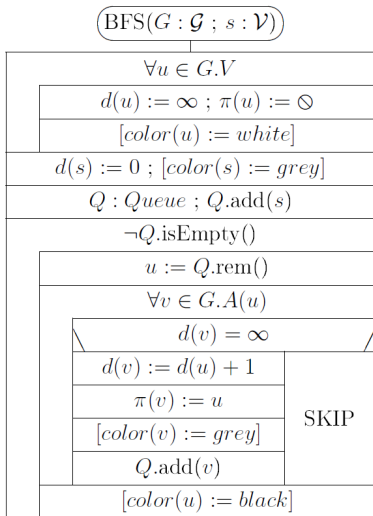
Directed Graph

	1	2	3	4	5	6	7
1	1	0	0	0	0	0	1
2	1	1	1	1	1	1	1
3	1	0	1	0	0	1	1
4	1	1	1	1	1	1	1
5	1	1	1	1	1	1	1
6	1	0	0	0	0	1	1
7	0	0	0	0	0	0	1

Transitive Closure

How can we calculate this matrix T ?

First solution: perform a BFS from each vertex as source. We do not need to compute distances, nor parent vertices, but only reachability. In BFS a vertex is reached if it is not white.

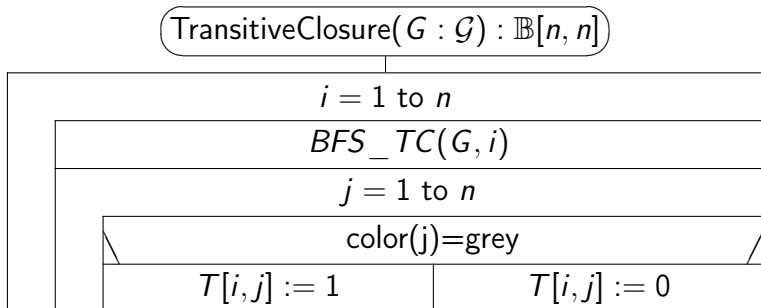


After running algorithm BFS_TC with source vertex i we have

$$T[i,j] = 1 \iff \text{color}(j) = \text{grey}$$

We obtain row i of matrix T .

If we run this algorithm for all vertices i , then we obtain T .



The time complexity will then be n times $\Theta(m + n)$, which is

$$\Theta(n(n + m))$$

For sparse graphs this is $\Theta(n^2)$, but for dense graphs it is $\Theta(n^3)$.

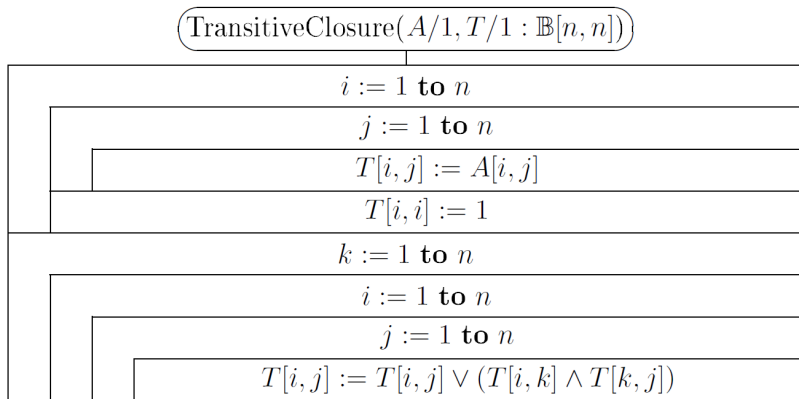
Another way of calculating transitive closure:

Suppose the graph is represented with adjacency matrix

$$A/1 : \mathbb{B}[n, n].$$

Now $\mathbb{B} = \{0, 1\}$ and we will use Boolean operators "AND" and "OR" (as $\wedge$ and $\vee$) on this set.

Another way of calculating transitive closure:

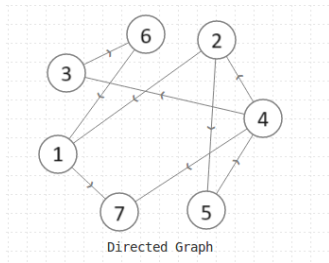


Clearly the time complexity for this algorithm is $\in \Theta(n^3)$.

Why is this algorithm good for computing Transitive Closure?

Notation. Let $i \overset{k}{\rightsquigarrow} j$ denote a path from vertex i to vertex j which uses only vertices $\leq k$ as inner vertices.
 ($i > k$ and/or $j > k$ is possible.)

So the path can be $\langle i, u_1, u_2, \dots, u_n, j \rangle$ where $u_1, u_2, \dots, u_n \leq k$.



In this graph there is no $2 \overset{1}{\rightsquigarrow} 3$ path, nor $2 \overset{4}{\rightsquigarrow} 3$ or $2 \overset{0}{\rightsquigarrow} 3$ paths, but there is $2 \overset{5}{\rightsquigarrow} 3$ path. ($i \overset{0}{\rightsquigarrow} j$ is a path that has no inner vertices, i.e. it is an edge.) There is no $3 \overset{5}{\rightsquigarrow} 1$ path, but there is $3 \overset{6}{\rightsquigarrow} 1$ path.

We will build a series of matrices: $T^{(0)}, T^{(1)}, T^{(2)} \dots$
 $T^{(n)} \in \mathbb{B}[n, n]$ with the following definition:

Definition.

$$T^{(k)}[i, j] = 1 \iff \exists i \overset{k}{\rightsquigarrow} j$$

where $k \in 0 \dots n$ and $i, j \in 1 \dots n$

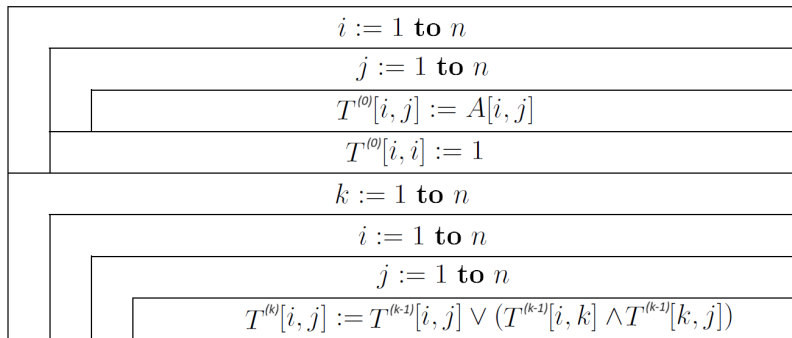
For every $k = 0, 1, \dots, n$ we will compute matrix $T^{(k)}$, and clearly

$$T^{(n)} = T.$$

Recursive relation among the T matrices:

- ▶ $T^{(0)} = A + I$
 - ▶ So $T^{(0)}$ is almost the same as the adj. matrix, but the main diagonal is changed to all 1's.
- ▶ $T^{(k)}[i,j] = T^{(k-1)}[i,j] \vee (T^{(k-1)}[i,k] \wedge T^{(k-1)}[k,j])$
 - ▶ There is a $i \overset{k}{\rightsquigarrow} j$ path if and only if
 - ▶ either there is an $i \overset{k-1}{\rightsquigarrow} j$ path
 - ▶ or there are paths $i \overset{k-1}{\rightsquigarrow} k$ and $k \overset{k-1}{\rightsquigarrow} j$

Use these properties to prove that the following algorithm produces the matrix $T = T^{(n)}$.



Do we really need all the $T^{(k)}$ matrices? Space complexity is $\in \Theta(n^3)$.

No, we don't.

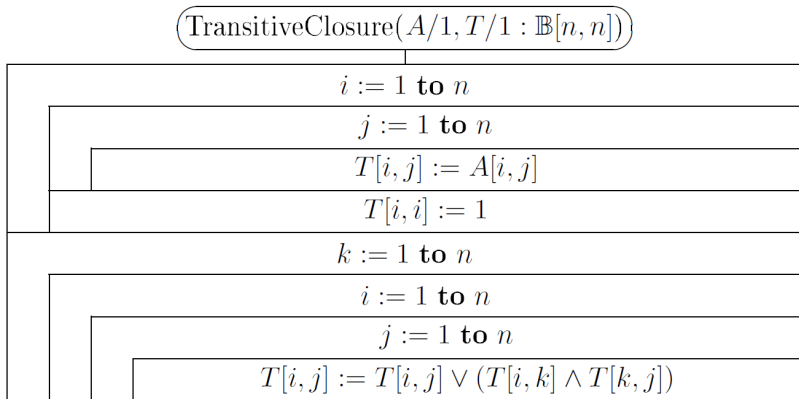
Observation: From the recursive relations we have:

- ▶ $T^{(k)}[i, k] = T^{(k-1)}[i, k]$
- ▶ $T^{(k)}[k, j] = T^{(k-1)}[k, j]$

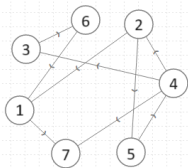
This means that column k and row k of matrix $T^{(k)}$ are the same as the appropriate column and row of matrix $T^{(k-1)}$.

From this it follows that a single matrix T is enough for the whole computation, because $T^{(k)}[i, j]$ depends only on $T^{(k-1)}[i, j]$, $T^{(k-1)}[i, k]$, $T^{(k-1)}[k, j]$.

So this algorithm will also calculate the transitive closure:



Clearly the time complexity for this algorithm is $\in \Theta(n^3)$,
space complexity is $\in \Theta(n^3)$.



Directed Graph

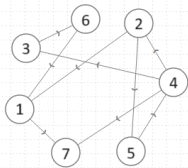
	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							



Directed Graph

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

	1	2	3	4	5	6	7
1	1	0	0	0	0	0	1
2	1	1	1	1	1	1	1
3	1	0	1	0	0	1	1
4	1	1	1	1	1	1	1
5	1	1	1	1	1	1	1
6	1	0	0	0	0	1	1
7	0	0	0	0	0	0	1

Transitive Closure